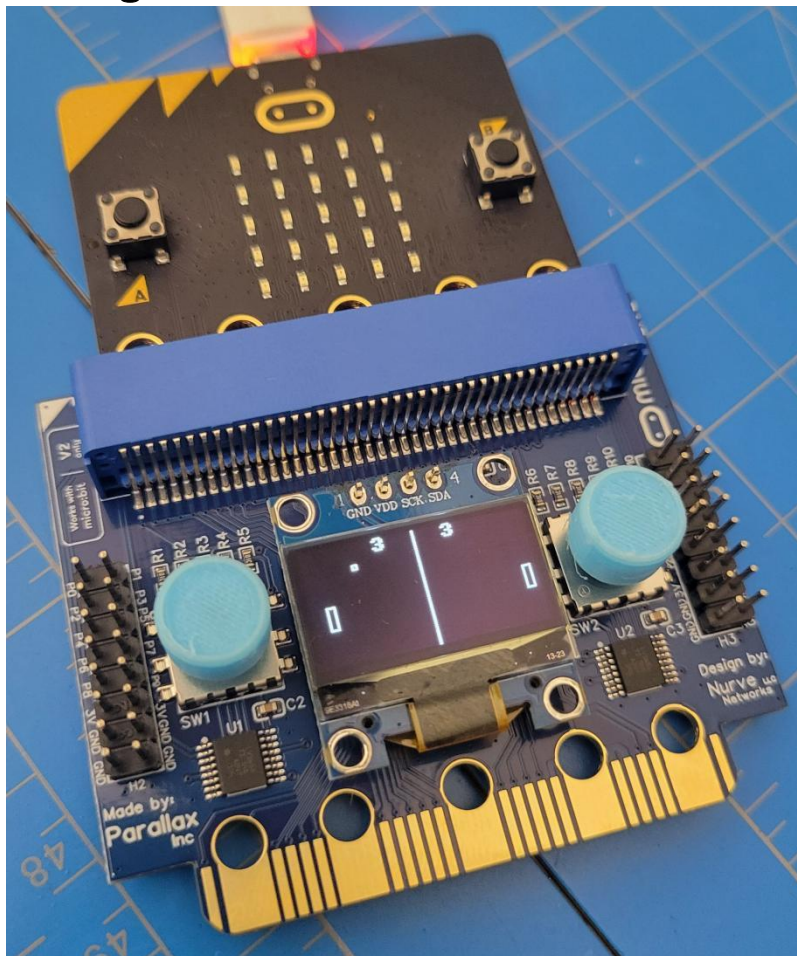


Nurve Networks LLC | www.nurve.net | support@nurve.net | Austin, TX 78732

Micro:bit OLED Badge

Programmers Guide and Datasheet



* micro:bit not included.

The **Micro:bit OLED Badge** supports a 128x64 monochrome bitmap OLED display, two serialized 5-way joysticks (Up, Down, Left, Right, and Fire), and interfaces directly to a micro:bit (v2 or V1). The adapter also passes through the entire micro:bit signal interface to the bottom of the adapter, so other compatible edge-connector micro:bit add-on boards can be plugged in.

Additionally, the entire I/O interface and power signals are broken out on (2) 2x8 0.1" headers on either side of the adapter to make prototyping and builds easier.

The 128x64 OLED is controlled by the SSD1306/1315 display controller and is communicated to via I2C (address 0x3C) from the micro:bit host. There are numerous Micro Python based display drivers available on the Makecode micro:bit Extensions repository as well as GitHub (see **External Resources**).

The Micro:bit OLED Badge draws 1-25 mA (depending on display settings) and is powered directly from the micro:bit host via the 3.3V supply bus. The micro:bit itself can be powered by USB, or battery. The badge will run for days on (2) AAA cells in the battery pack.

The joysticks are simple on/off momentary switches that are serialized via a pair of 74LV165A parallel to serial shift registers and read through a single GPIO pin (P16) and controlled by two other GPIOs (P8,P9) , so their impact on the GPIO resources is minimal.

Features

- BBC micro:bit plugs directly into the OLED Badge and powers it from the 3.3V supply.
- 128x64 1-bit monochrome bitmap OLED display.
- SSD1306/1315 I2C display driver interfaced via I2C (address 0x3C, or 0x3D).
- 5-way joysticks with serialized data interfaced via 3 GPIO pins (with removable button caps).
- 2x8 0.1" breakout headers that export all micro:bit GPIOs as well as power and ground.
- Pass-through micro:bit edge interface at bottom of badge adapter, so the badge can plug into other micro:bit compatible boards and be "daisy-chained".

NOTE

Works with micro:bit v2 or v1, although GPIO P9 is used by the OLED adapter for clock, and during joystick reads may disrupt 5x5 LED matrix on micro:bit v1 models.

Key Specifications

- Draws 1 - 25mA of current, less than 1uA when the display is powered down.
- Compatible with both BBC micro:bit v2 and v1.
- Operating temperature: -4 to +185 °F (-40 to +85 °C).
- Dimensions including edge connector: 66.9 x 58.9 mm (2.64 x 2.31 in)

Application Ideas

- Electronic badge.
- Games and media applications.
- Animated information display for sensors and state.
- Weather stations and monitoring applications.
- Graphics and text applications.

Resources and Downloads

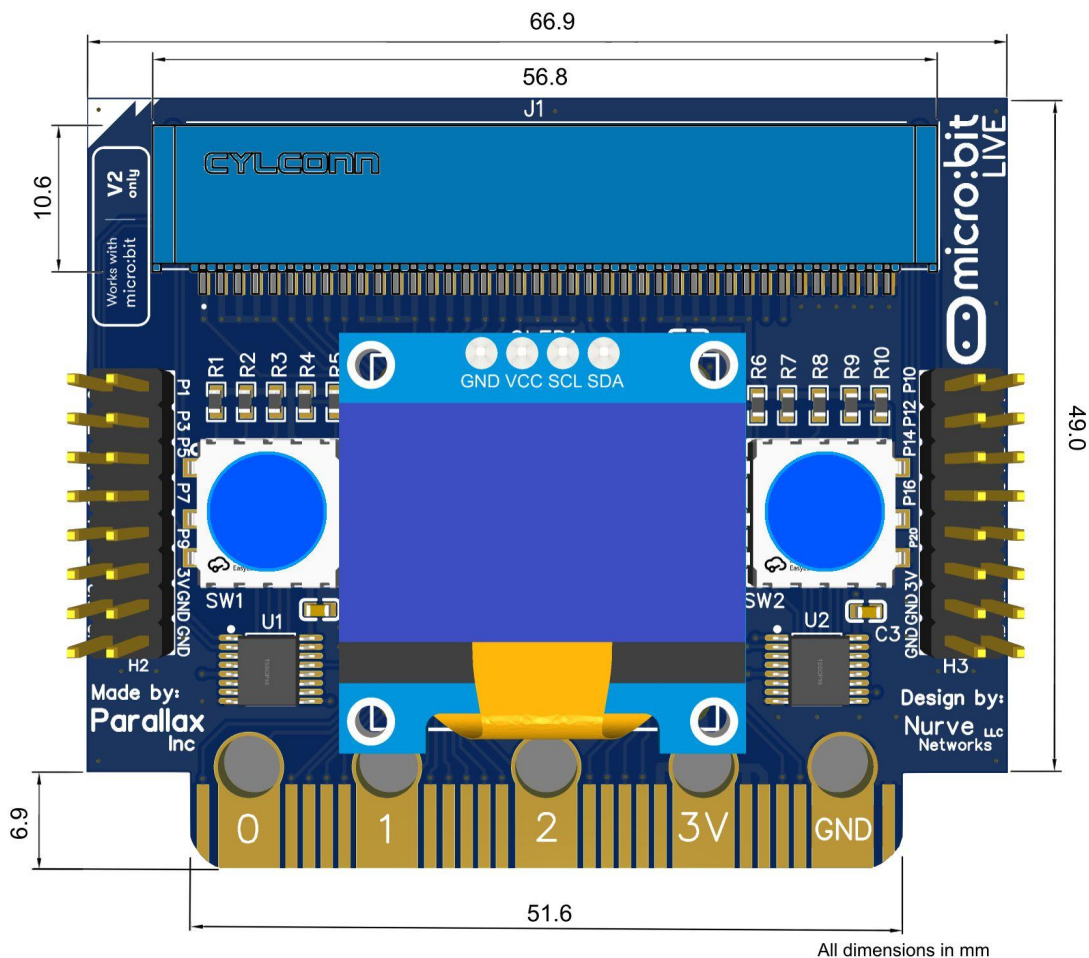
Software demos, drivers, and other resources can be found on the public GitHub at:

<https://github.com/NurveNetworksLLC/MicrobitOLEDBadge>

1.0 - Micro:bit OLED Badge Overview

Figure 1.0 below is a mechanical rendering of the micro:bit Electronic Badge. The badge is driven and controlled by a BBC micro:bit that plugs into the top 40-pin interface (J1). The badge derives its power from the 3.3V supply from the micro:bit (either USB or battery powered).

Figure 1.1 - Micro:bit OLED Badge PCB Annotated.



The badge can be used for a number of applications, such as displaying informational text, graphics, sensor outputs, or gaming. The OLED is a bitmapped 128x64 monochrome display with extremely high contrast and brightness. The OLED display can be used to draw very detailed graphics (with a number of available drivers), simple text in varying sizes, or animated imagery for simple games. The OLED is driven by an internal SSD1306/1315 display driver, so you don't need to write a driver to use the badge; however, if you are interested in doing so, the datasheet can

be found in the **External Resources** section of this document, and there are 100s of resources and examples on the internet on how to write drivers for this very popular OLED display controller.

The OLED display is controlled via the micro:bit I2C bus over pins (P19 - SCL, P20 - SDA). To draw on the display, I2C commands are issued by a 3rd party driver. Most drivers support text and graphics. Some are better at text than graphics and vice-versa. Later in this document, a number of options and demos show how to display text and graphics on the OLED with a few lines of code.

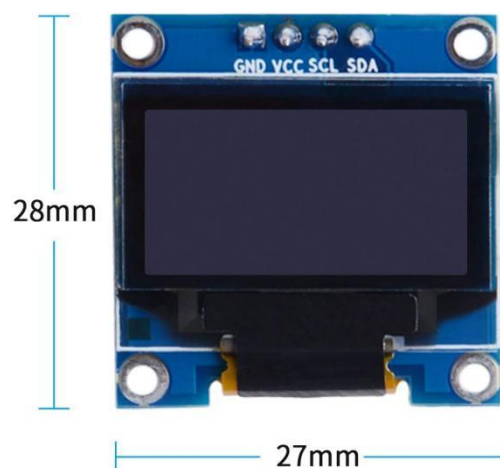
In addition to the OLED, the badge also has two momentary switch based 5-way joysticks (SW1, SW2). These joysticks have a total of (10) momentary switches. Normally, reading all (10) switches would use up nearly half of the available GPIO pins on the micro:bit. Therefore, instead of connecting each switch to a GPIO, the switches are latched into two 8-bit 74LV165A parallel to serial shift registers, using only (3) GPIO pins. The joystick state bits are latched and shifted into the micro:bit. MicroPython and demo samples show how to do this later in the document.

Finally, the Micro:bit OLED Badge has generous breakout headers H2, and H3, on either side of the adapter. These 2x8 0.1" pitch headers break out all the signal interface GPIOs, power, and ground for you to use to connect other hardware. Additionally, a "clone" edge connector of the BBC micro:bit is at the bottom of the adapter, so you can "daisy chain" the Micro:bit OLED Badge and plug it into other 3rd party add-on boards.

2.0 - Programming the 128x64 Pixel OLED

The OLED on the Micro:bit OLED Badge is a 128x64 pixel monochrome OLED on break out board that is attached to the badge PCB. The OLED is shown below in Figure 2.1.

Figure 2.1 - The 128x64 Pixel Monochrome OLED Display with SSD1306/1315 Controller and I2C Interface.



As mentioned in the overview paragraphs above, the OLED is mounted on another PCB, this allows us to use different OLED boards in the future, as long as they are I2C interfaced and have roughly the same mechanical dimensions. For example, we could swap out the monochrome OLED for an RGB version for version 2 of the micro:bit OLED Badge! Anyway, for now, we have the very popular 128x64 pixel OLED that is driven by an on-board SSD1306/1315 display controller; the datasheet(s) can be found here:

[https://soldered.com/productdata/2022/03/Soldered SSD1306 datasheet.pdf](https://soldered.com/productdata/2022/03/Soldered_SSD1306_datasheet.pdf)

[https://soldered.com/productdata/2022/03/Soldered SSD1315 datasheet.pdf](https://soldered.com/productdata/2022/03/Soldered_SSD1315_datasheet.pdf)

The OLED on your board may have the SSD1306 or SSD1315 display controller. They are 99% identical, with the SD1315 supporting larger displays, but we are using the SSD1306 functionality, which is cloned with the SSD1315 and a subset. That said, if you peruse the datasheet(s) you will see the display controller IC are very complex, and writing a driver for them is quite a daunting task. However, luckily, the display controller IC can be communicated with via I2C (Inter IC Communications) which is supported by the micro:bit's processor natively and is exported out via P19, and P20 for SCL, and SDA respectively.

If you're not familiar with the inner workings of I2C then in short, it's a serial (2) wire protocol (clock and data) along with ground, that allows up to 128 devices to be on the same bus and each addressed with a unique address. The protocol can clock at various speeds from 100 kBits/s to 5 Mbits/s as shown in Table 2.1 below.

Table 2.1 - I2C Latest Speed Modes.

Speed Mode	Maximum Speed
Standard Mode (Sm)	100 kbits/s
Fast Mode (Fm)	400 kbits/s
Fast-mode Plus (Fm+)	1 Mbits/s
High-speed Mode (Hs-mode)	3.4 Mbits/s
Ultra-fast Mode	5 Mbits/s

Of course, how fast you can clock the I2C bus depends on the host hardware and the device you are communicating with. Nonetheless, most modern devices support at least the "Fast Mode" at 400 kbits/s.

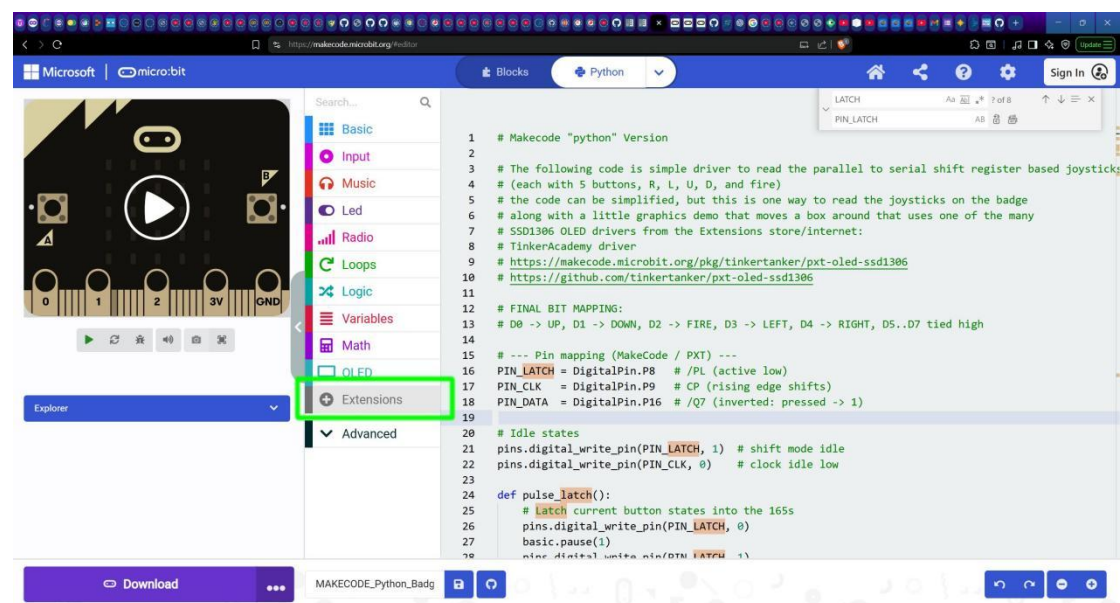
NOTE

If you use a 3rd party driver and it acts unpredictable -- if the driver supports controlling the signaling speed of the driver, try slowing it down.

2.1 Makecode Extension Based Drivers for OLED

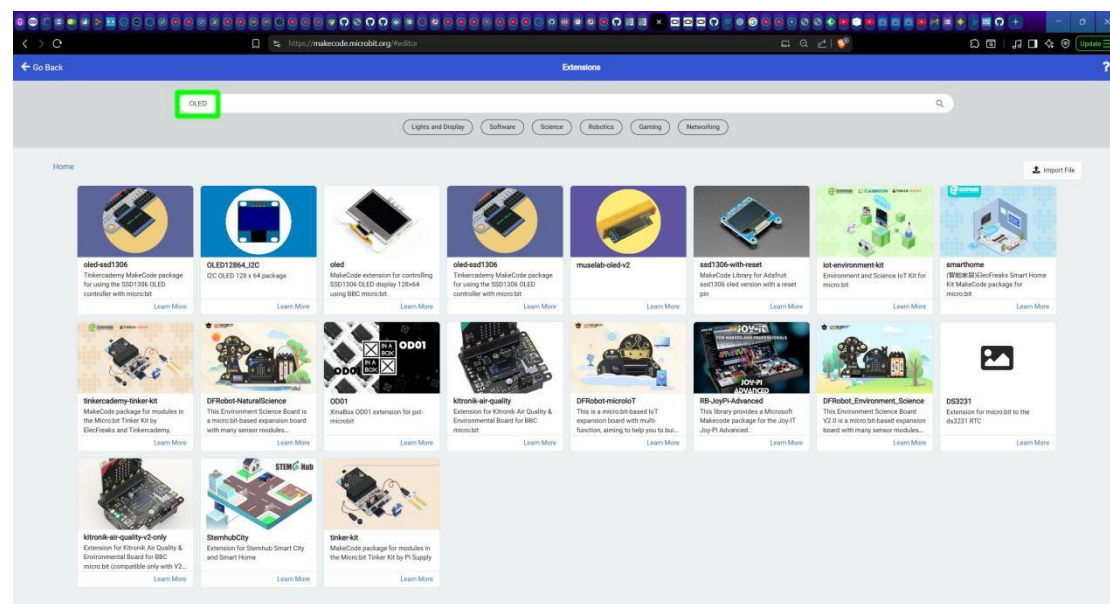
We are going to use a 3rd party driver for the OLED itself (we are working on an ultra high performance driver that will be released in the near future). So, where do you find these drivers for the micro:bit and the 128x64 OLED with SSD1306/1315 driver IC? Well, they are on GitHub, on various embedded development sites, and they can be found in the Makecode Extensions store by searching the **Extensions** store by clicking the Extensions tab (highlighted in green in figure) when working with the micro:bit Makecode IDE, as shown in Figure 2.2 below.

Figure 2.2 - Accessing the Extensions Store from the Makecode IDE.



When you click the Extensions tab, it takes you to the Extensions store, which allows you to search for anything you want: servos, motors, NEO pixels, and so forth. The OLED drivers can be found by searching for “OLED” or “SSD1306 OLED” seem to work best as reasonable searches -- your mileage may vary. Figure 2.3 below illustrates a search when this document was generated and shows what extension drivers are available.

Figure 2.3 - Searching for SSD1306/1315 Extensions to Control OLED.



This is the process you will need to go through to find drivers for the 128x64 OLED with SSD1306/1315 display driver. The drivers usually support both text and graphics, with varying levels of features (and bugs). There are (3) extensions that seem to work with varying levels of success; they are:

Kitronik OLED Extension

<https://makecode.microbit.org/pkg/kitronikltd/pxt-kitronik-128x64display>

XinaBox OD01 OLED Extension

<https://makecode.microbit.org/pkg/xinabox/pxt-od01>

Tinker PXT OLED Extension

<https://makecode.microbit.org/pkg/tinkertanker/pxt-oled-ssd1306>

In all cases, these drivers are designed to drive a 3rd party product that uses the 128x64 pixel SSD1306/1315 OLED display driver, so we can use them as well. Each driver is mostly functional, but you will find that each has different strengths and weaknesses; this is why we are developing a high-performance driver for the future. But, for now, these drivers are more than capable of displaying text and graphics. I suggest you review the documentation and API for each with the links above. Since these are Makecode Extensions you can use each Extension with block code, Python, or JavaScript, although some documentation leans toward one language or another.

You will see any additional demos and examples of each in the **Demos and Examples** section of this document, but for now, let's take a look at one of them that has a lot of text-related features (XinaBox OD01 OLED Extension) and build a simple text application with it so you can see the general workflow using an extension-based driver in Makecode.

2.1.1 Working with the XinaBox OD01 OLED Extension

The first step in using any of these drivers or extensions is to review the documentation. In general, the quality of the documentation is hit and miss on the Makecode Extensions store. These OLED drivers are no different, so the trick is to review the documentation, then include the extension in your program, and hopefully, between the documentation on the extension page:

<https://makecode.microbit.org/pkg/xinabox/pxt-od01>

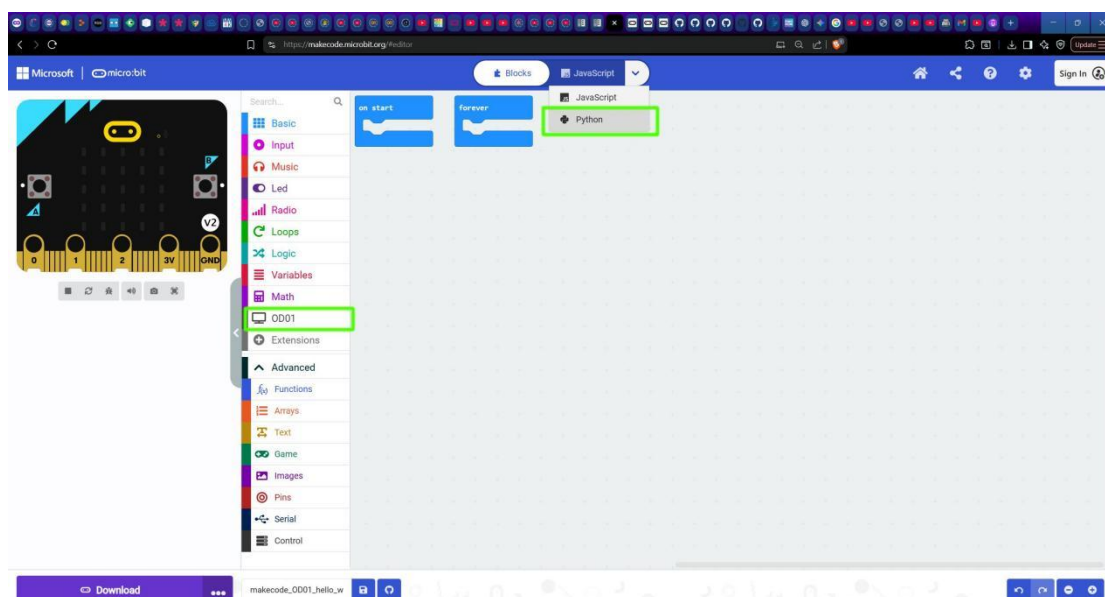
And the “**code completion**” help for the extension, you can figure out how to use the extension. In this case, I have found that the extension has a number of functions mentioned in the documentation, but no examples, as well as some undocumented functions. Anyway, let’s begin with the OD01 driver extension and see some examples.

Step 1 - Create a new empty Makecode project supporting Python in the makecode.microbit.org editor.

Step 2 - Navigate to the **(+) Extensions** tab, click it, and search for “OLED”. Find the **Xinobox OD01 driver**, click on it, and then it will load into your project as shown in Figure 2.3 above. The driver is in the 3rd column, 2nd row down, that’s the one you want.

Step 3 - The extension will download for a moment, and then you will return to the IDE where your blank project was. Now, you should see that the new extension OD01 is in your programming palette as shown in Figure 2.4 below. Finally, select Python as the target language. The editor should transcode/transpile the block language into Python, and you are all set to play with some OLED driver code.

Figure 2.4 - The XinaBox SSD 1306/1315 OLED Driver Loaded and Python Selected.



2.1.2 The OD01 Driver API and Demos

The OD01 OLED driver was written years ago, but it still works and is a good example that supports block coding as well as Python and Javascript. Additionally, it has some text zooming features that come in handy. Let's start off by writing a "Hello World" text program.

Referring to the documentation, the OD01 driver requires very little to get it up and running. The extension creates the default object "OD01", you can actually just start making calls with it, you don't have to initialize anything, create an object, etc. It's already done for you. With that in mind, let's review the API functions.

```
# Turn the display on/off, flag = (True, False).
OD01.display( flag )

# Clears the screen.
OD01.clear()

# Inverts the screen based on flag = True, False.
OD01.invert( flag )

# Displays text at pos (col, row) with color = 0,1.
OD01.show_string(string, col, row, color)
```

```
# Displays number at pos (col, row) with color = 0,1.
OD01.show_number(number, col, row, color)
```

These next (2) text printing functions print as if the screen is a terminal; hence, no coordinates are needed to indicate where to print. They are nice if you just want to print to the screen, and let it handle where to print next like a typewriter.

```
# Prints string at next position.
OD01.print_string(string, True)

# Prints number at next position.
OD01.print_number(number, True)

# Draw a pixel at (x,y) with color = 0,1.
OD01.pixel( x,y, color)

# Draw an h-line from (x1,y) to (x2,y) with color = 0,1
OD01.horizontal_line(x1, x2, y, color)

# Draw an v-line from (x,y1) to (x,y2) with color = 0,1
OD01.vertical_line(y1,y2,x,color)

# Draw an rect at (x,y), width x height with color = 0,1
OD01.rectangle(x,y,width, height)
```

These next functions are not documented in the documentation, but the extension has them nevertheless. They control text scaling, and when you call them, they set the **“state”** of the text size/scale for all text drawn until you change the size and/or scale again.

```
# Sets the text to size 1, small.
OD01.set1_x()

# Sets the text to size 2, medium.
OD01.set2_x()

# Zooms in/scales the text rendering.
OD01.zoom_in()

# Zooms out/scales the text rendering.
OD01.zoom_out()
```

Now that you have seen the API for the OD01 driver extension, let’s write some code and download it to the micro:bit! Here’s the most minimal program possible for “Hello World” take a look at the code snippet below.

Code Snippet 2.1 - Bare Minimum “Hello World” with OD01 Driver Extension.

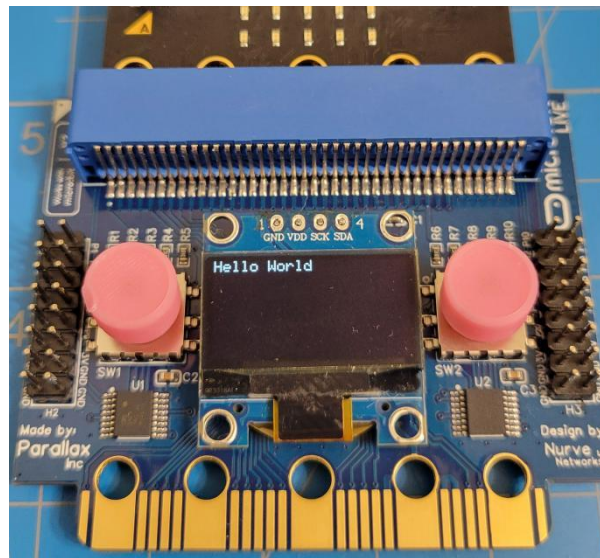
```
# print hello world on screen with
OD01.print_string("Hello World", True)

# sit in loop forever...
def on_forever():
    pass
basic.forever(on_forever)
```

Download this to your micro:bit with the OLED badge plugged in, and you should see some tiny text on the OLED as shown in Figure 2.5 below. A copy of the code is located here as well:

`\\Demos\\OD01_Demos\\microbit-makecode_OD01_hello_world_01.hex`

Figure 2.5 - The Bare Minimum “Hello World” Program for the ODO1 OLED Driver.



As you can see, the text is quite small! This is great if you want to display a lot of information, or even show a listing, program, or some text to read. But, if you want to be able to “see” the text from a few feet away, then we need a way to scale the text. Fortunately, the ODO1 driver has some text scaling functions. It allows you to select 1X or 2X font sizes with the **set1_x()** and **set2_x()** functions respectively. Additionally, we can zoom in and out with **zoom_in()** and **zoom_out()**. The next code snippet 2.2 illustrates using these functions as well as calling the **display()** function to make sure the display is on.

Code Snippet 2.2 - Scaling the “Hello World” Text with ODO1 Driver Extension.

```
# turn ON display, this is default, but doesn't hurt
OD01.display(True)

# set text size 1 or 2
OD01.set2_x()

# set zoom level in or out
OD01.zoom_in()

# clear the display in case of garbage on screen
OD01.clear()

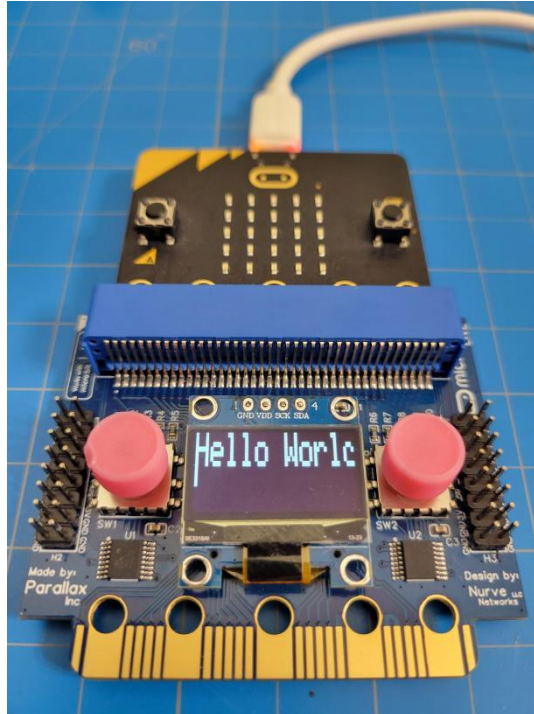
# print hello world on screen with
OD01.show_string("Hello World", 0, 0, 1)

# sit in loop forever...
def on_forever():
    pass
basic.forever(on_forever)
```

Download this program to your micro:bit with the OLED badge plugged in, and you will see something like the image shown in Figure 2.6 below. A copy of the code is located here as well:

`\Demos\OD01_Demos\microbit-makecode_OD01_hello_world_02.hex`

Figure 2.6 - Scaled Text “Hello World” Program.



If you look closely at the image, you will see a slight anomaly on the right-hand edge of the screen. Part of the “d” character is missing; it has been drawn on the other side of the screen. This is due to the fact that these drivers rarely have “clipping” algorithms, and if you try to draw or print something too big, it just draws it and runs over the boundaries in memory. To remedy this, we could draw less text in this case like “HelloWorld” and get rid of the space between words, or another option is to set the font smaller or zoom back out. The point is that when drawing, you must be careful of scale and clipping issues with these drivers in general.

For the next example of this driver, let’s try some graphics programming and use the ***rectangle()*** functions to draw randomly on the screen, and clear the screen off every cycle. Snippet 2.3 below draws random rectangles on the OLED.

Code Snippet 2.3 - Code to Draw Random Rectangles on the OLED Screen.

```
# turn ON display, this is default, but doesn't hurt
OD01.display(True)
OD01.print_string("I see Rectangles...", True)
basic.pause(1000)

# sit in loop forever...
def on_forever():
    # clear the display in case of garbage on screen
    OD01.clear()

    # draw 100 rects at end
    for i in range(0,100):
        x = randint(0, 128-16)
        y = randint(0, 64-16)
        width = randint(2, 16)
        height = randint(2,16)

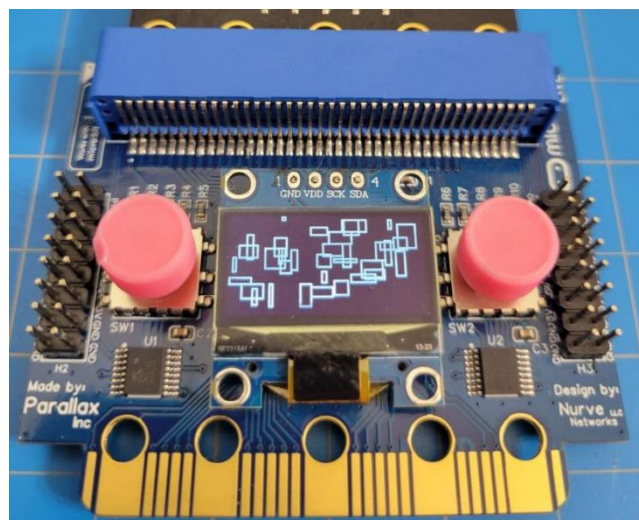
        OD01.rectangle(x, y, x+width, y+height, 1)

basic.forever(on_forever)
```

If you type (or copy and paste) in the program and download it to your micro:bit with the OLED badge, you should see something like what's shown in Figure 2.7 below. There is also a copy of the code located in the master .ZIP from the Downloads:

`\Demos\OD01_Demos\microbit-makecode_OD01_rectangles_01.hex`

Figure 2.7 - Drawing Random Rectangles with the OD01 OLED Driver.



That wraps it up for the OD01 driver examples. You can find most of the examples shown in the code snippets detailed in the **Demos and Examples** section of the document. Also, there is one final demo that scrolls text that you might want to

check out named **microbit-makecode_OD01_text_demo_01.hex** it scrolls text by using string operations like splicing and some other tricks, so make sure to check it out.

2.2 GitHub and Embedded Site-Based Driver Options

In this section, let's review other options that you can use to drive the 128x64 OLED. If you google and or search on GitHub, you can find many classic MicroPython-based drivers for the 128x64 SSD1306/1315; however, many are for other platforms such as Arduino, ESP32, or Raspberry Pi, so you need to be careful in your searches to make sure that you have found a micro:bit compatible driver. I have found a few pure MicroPython drivers on the internet and GitHub, which we will look at now.

Core Electronics Driver

Comments: One of the best overall drivers. Has a lot of functional graphics features. The Atari "Pong!" clone starter game shown in the Demos and Examples section was written with this driver.

<https://core-electronics.com.au/guides/micro-bit/piicodev-oled-ssd1306-microbit-guide/>

<https://github.com/CoreElectronics/CE-PiicoDev-SSD1306-MicroPython-Module/tree/main>

Fizban99 Driver

Comments: This is the fastest driver, and supports bitmaps ("stamps") and has tools to create them, and store them (all detailed on the GitHub and docs).

https://github.com/fizban99/microbit_ssd1306

2.3 Using the Core Electronics OLED Driver

In this **Demos and Examples** section later in this document, you will see examples using most of the drivers to help you get started. That said, there are too many drivers to cover them all, so we are going to focus on showing a detailed walkthrough for one of the **Extension** based **Makecode** drivers, as well as a pure MicroPython-based example, using both the Core Electronics driver and the Fizban99 driver. Between these tutorial examples, you will see the steps you need to use any driver. Let's start with the **Core Electronics** driver. The driver and documentation can be found on the website below.

<https://core-electronics.com.au/guides/micro-bit/piicodev-oled-ssd1306-microbit-guide/>

Navigate to the website and download these (3) files:

(1) PiicoDev Unified Library: **PiicoDev_Unified.py**

https://raw.githubusercontent.com/CoreElectronics/CE-PiicoDev-Unified/main/min/PiicoDev_Unified.py

(2) The device module: **PiicoDev_SSD1306.py**

https://raw.githubusercontent.com/CoreElectronics/CE-PiicoDev-SSD1306-MicroPython-Module/main/min/PiicoDev_SSD1306.py

(3) The text font files: **font-pet-me-128.dat**

<https://github.com/CoreElectronics/CE-PiicoDev-SSD1306-MicroPython-Module/raw/main/font-pet-me-128.dat>

The files can also be found on there GitHub here:

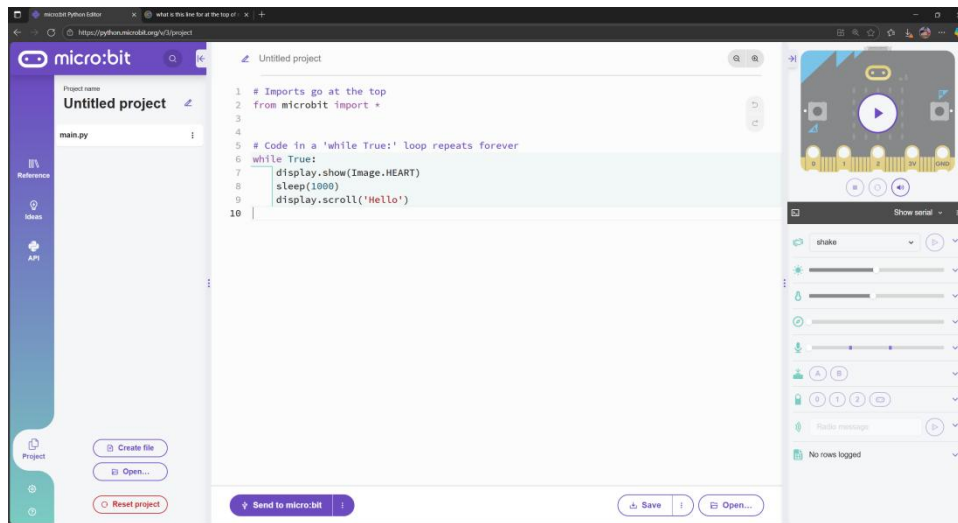
<https://github.com/CoreElectronics/CE-PiicoDev-SSD1306-MicroPython-Module/tree/main>

For your convenience, these 3 files have also been uploaded to the Github, and you can find the single file **PiicoDevDriverSSD1306.zip** in the inside the master **Demos.zip** folder. Once you have the 3 files, then you are ready to build your first program in MicroPython. Referring to the Core Electronics documentation on their website above, we are going to write a few simple programs with the micro:bit MicroPython IDE, so first make sure you have your micro:bit plugged into the your micro:bit OLED Badge, and its connected to your PC via a USB cable. Then launch or navigate into the micro:bit MicroPython editor IDE in your browser:

<https://python.microbit.org/>

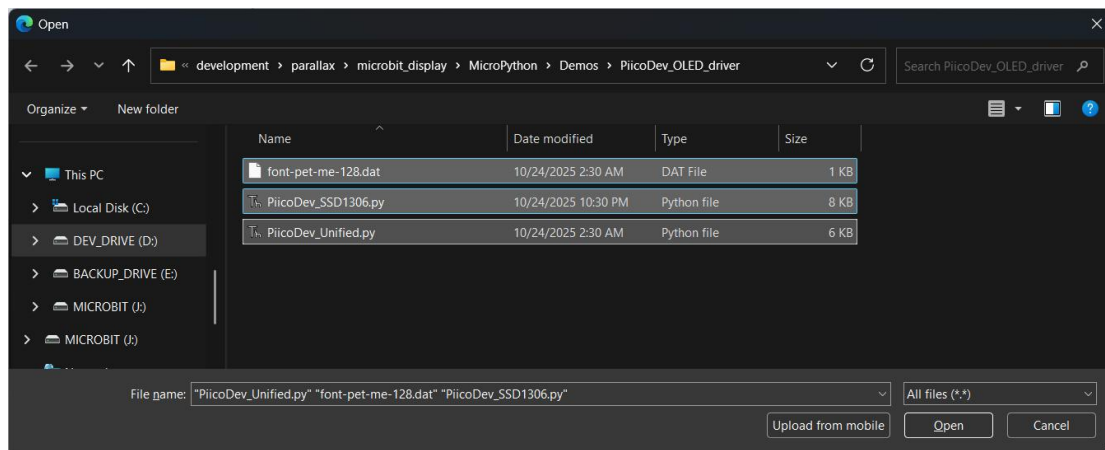
Once on the page, click the **Reset** the project button to start a fresh project, and you should see something like that shown in Figure 2.4 below.

Figure 2.4 - micro:bit MicroPython after Reset.



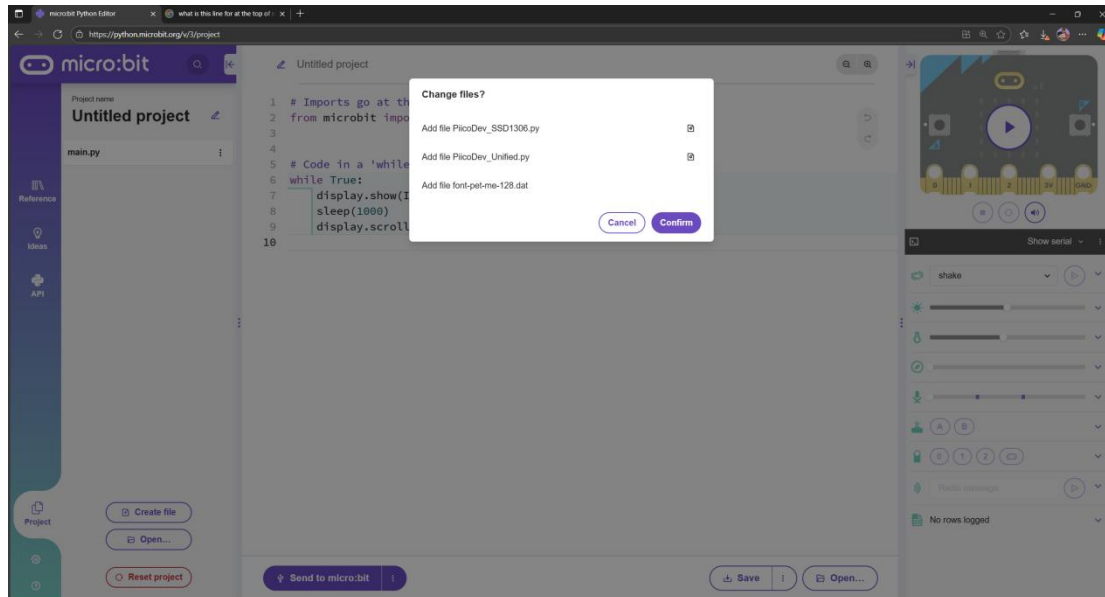
Next, you will need to make sure that your micro:bit is connected to the application by clicking the 3 dots icon ... on the purple **Send to micro:bit** button located at the bottom of the code editor. Once you have connected to your micro:bit, you are ready to add the library files for the Core Electronics PiicoDev OLED driver. To do this, click on the **Open...** button in the left **Project** panel, then navigate to where you have stored the files and decompressed them, select all of them, and click Open on the dialog as shown in Figure 2.5 below:

Figure 2.5 - Adding the Core Electronics, PiicoDev SSD1306 Driver Files to Project.



When you do this, the editor is going to put up a dialog asking you if you want to change replace the main file, click the little **paper/gear icon** next to each file and select **Add** (not replace), do this for all files, then click Confirm on the dialog. Basically, we want to add these files to the project as libraries, but we want to keep the little starter **main.py** for now. This is shown in Figure 2.6 below:

Figure 2.6 - Adding the Library Files to the Project.



Now, you are all set! Go ahead and name your project something like **CoreElectronicsPixelTest_01** or whatever suits you. And let's do some graphics programming.

The first thing you need to do on each program is import the new library, so delete the starter code (that displays a heart on the LED display), and start with this always, as shown below:

Code Snippet 2.4 - Importing the CoreElectronics / PiicoDev Library Module.

```
# import the microbit libraries
from microbit import *

# import everything from the PiicoDev library
from PiicoDev_SSD1306 import *
```

Now, let's write a simple program that uses some of the library functions from the driver. The driver has a lot of functionality; it can draw text, points, lines, images, and much more. You can try out the API functions by following the tutorial on the Core Electronic's website to learn about every function. But, the general idea is that you must initialize the driver, then clear the screen, draw whatever you wish then "show" the screen and repeat. Here's the complete program below that performs all these steps and draws a set of random pixels on the OLED, waits 3 seconds then clears the screen and does it again. We are going to use the **rect()**, and/or **fill_rect()** API function(s) to draw rectangles:

```
rect(x,y,width,height,color)
fill_rect(x,y,width,height,color)
```

Where colour is 0 (black) or 1 (white).

Also, we need the `fill( colour )` function that fills the screen with black or white:

```
fill( color )
```

And finally, we need the `show()` function, which presents the frame buffer to the display and shows it.

```
show( )
```

It takes no arguments.

Putting all these steps together as well as creating a display object and initializing it, we have the code shown in snippet 2.5 below.

Code Snippet 2.5 - Simple Program that Draws Pixels on the OLED with the `fill_rect()` Function.

```
# import the microbit libraries
from microbit import *
from random import randint

# import everything from the PiicoDev library
from PiicoDev_SSD1306 import *

# initialize the driver and create a display object
display = create_PiicoDev_SSD1306()

# forever...
while True:
    # clear the screen each frame
    display.fill(0)

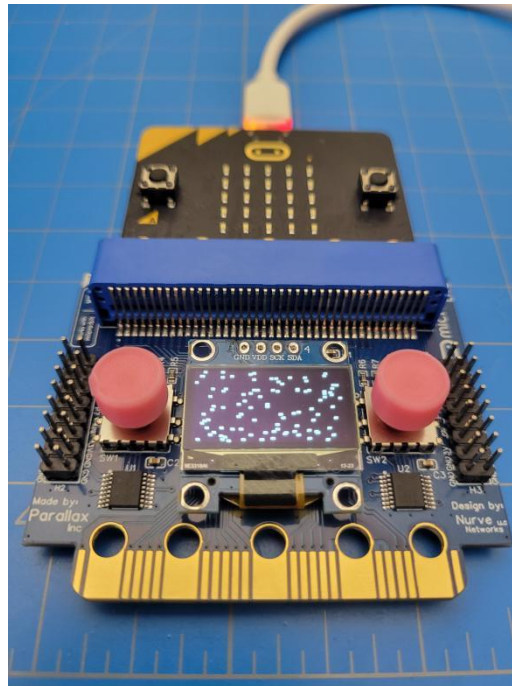
    for i in range(100):
        # generate a random location on screen
        x = randint(0, 127)
        y = randint(0, 63)

        # draw a small filled rectangle to look like a "star"
        display.fill_rect(x,y,2,2,1)

    display.show()
    sleep(100)
```

You should see something on your OLED screen that looks like the image in Figure 2.7.

Figure 2.7 - The Pixel Plotting Demo Running with PiicoDev Driver.



A copy of this program is located in the master source file .ZIP in the Downloads located here:

`\Demos\CoreElectronics\CoreElectronicsPixelTest_01.hex`

Next, let's try our hand at drawing text with the **`text()`** function.

```
text(string, x, y, colour)
```

string - is a python string.

x, y - are the top left coordinates.

colour - is 1 (white) or 0 (black).

Referring to the API docs for the driver, there are a number of overloaded text drawing functions, and there's even a sample program that shows how to draw text, numbers, literals, let's create a new project, call it **`CoreElectronicsTextTest_01`** and copy the source shown in code snippet 2.6 into it.

Code Snippet 2.6 - Text Rendering Program Using the text() Function.

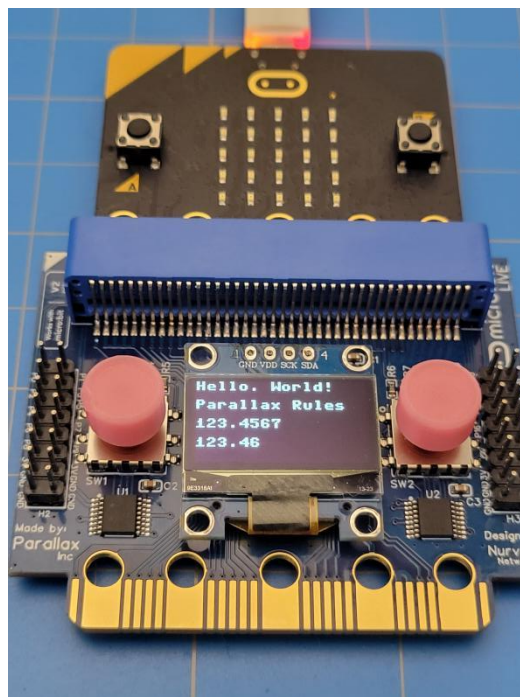
```
# import the microbit libraries
from microbit import *
from random import randint

from PiicoDev_SSD1306 import *
display = create_PiicoDev_SSD1306()
myString = "Parallax Rules"
myNumber = 123.4567

display.text("Hello, World!", 0,0, 1) # literal string
display.text(myString, 0,15, 1) # string variable
display.text(str(myNumber), 0,30, 1) # print a variable
display.text("{:.2f}".format(myNumber), 0,45, 1) # use formatted-print
display.show()
```

If you enter this program and upload it to your micro:bit OLED Badge, then you should see something like that shown in Figure 2.8 below.

Figure 2.8 - Text Rendering Demo Running.



As usual, you can find a copy of this program in the master Downloads .ZIP file in the following location:

`\Demos\CoreElectronics\CoreElectronicsTextTest_01.hex`

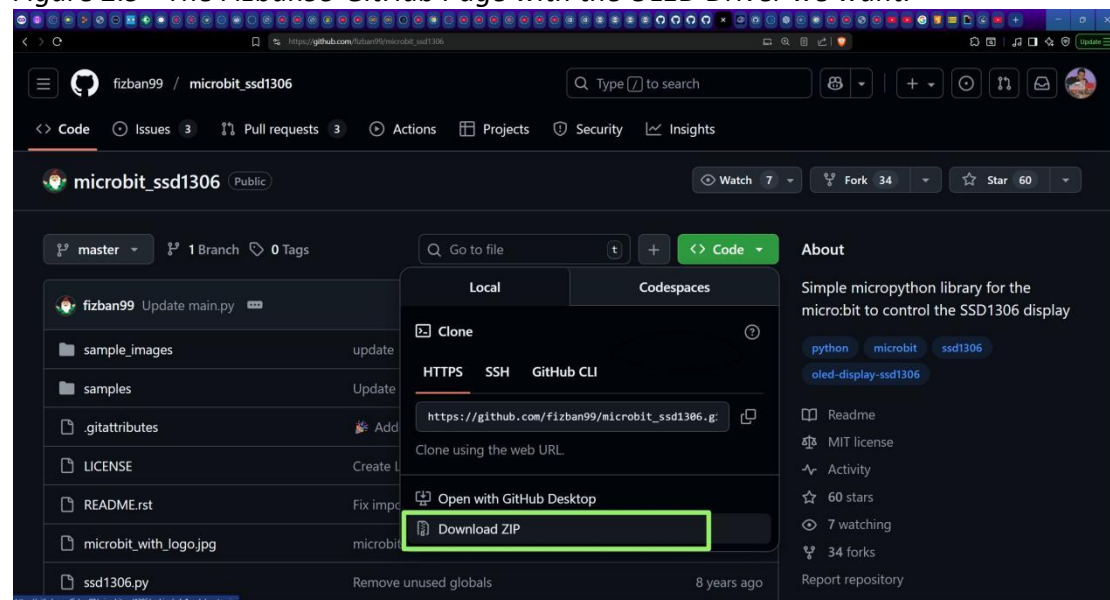
2.4 Using the Fizban99 OLED Driver (Advanced)

I saved the best for last! The **Fizban99** OLED driver is perhaps the highest-performance driver of the drivers reviewed. With it you can definitely make games that update the screen at a reasonable frame rate with a bit of work. The driver cheats a bit by reducing the effective resolution of the 128x64 pixel OLED screen to 64x32, thereby reducing the memory footprint to 25% of normal. However, but the downside is that each pixel is now 2x2 pixels. However, the full 128x64 pixel size is used by the **show_bitmap()** function, which loads a 1-bit bitmap and displays it. The reduction of resolution to 64x32 might seem like a problem, but honestly it works out great. Since pixels are so small, you can barely see a single pixel, so making the “working” pixels 2x2 makes no real difference, plus it speeds things up 400%. The driver supports text, graphics, “stamps” (small bitmaps you can define or use micro:bit LED images built in), along with the ability to display a full screen 1-bit image, as mentioned. The driver is located on GitHub here:

https://github.com/fizban99/microbit_ssd1306

If you’re not a guru in GitHub, basically it’s a code repository, so all we need is the .ZIP file with the driver, docs, etc. Take a look at Figure 2.9 below, click on the “<> CODE” button, and download the driver as a .ZIP. There is also a copy of this named **Demos\fizzban99_microbit_ssd1306_master.zip** in the GitHub ZIP micro:bit OLED Badge repo.

Figure 2.9 - The Fizban99 GitHub Page with the OLED Driver we want.



The first thing you will want to do is read the documentation on the **Fizban99** GitHub page itself; it’s complete and covers everything, albeit a bit rushed. Anyway, go ahead and peruse the docs on the page, and then return here.

In the **Fizban99** GitHub repo, you will find the following important files and folders:

11/16/2025 12:36 AM	<DIR>	samples
11/16/2025 12:36 AM	<DIR>	sample_images
11/16/2025 12:36 AM	42,461	microbit_with_logo.jpg
11/16/2025 12:36 AM	7,252	README.rst
11/16/2025 12:36 AM	2,258	ssd1306.py
11/16/2025 12:36 AM	307	ssd1306_bitmap.py
11/16/2025 12:36 AM	382	ssd1306_effects.py
11/16/2025 12:36 AM	215	ssd1306_img.py
11/16/2025 12:36 AM	569	ssd1306_px.py
11/16/2025 12:36 AM	1,092	ssd1306_stamp.py
11/16/2025 12:36 AM	629	ssd1306_text.py

2.4.1 Sample Games for the Fizban99 Driver

The **\samples** folder has a number of sample applications and games for the micro:bit; they all work more or less, but they don't support the joysticks, of course, on the OLED badge. So later in the document, we will rework one of them (an Asteroids type game) to support the joysticks. The games in the **\samples** folder are:

\asteroids - A port of another game called "Bitflyer" with Asteroids inspiration.
\heart_game - Tilting game with a heart on LED display.
\microbit_logo - Displays the micro:bit logo using ***show_bitmap()*** function.
\snake_game - Working version of Snake with tilt support to steer the snake.

So, to get any of these sample games to work, you need the main file found in each of the respective folders, along with any driver files the **main.py** program imports. Therefore, look at the imports at the top of each, and then make sure you include the driver files needed from the master list:

```
ssd1306.py
ssd1306_bitmap.py
ssd1306_effects.py
ssd1306_img.py
ssd1306_px.py
ssd1306_stamp.py
ssd1306_text.py
```

Each of these driver files has documentation at the top of the source if you want to know what they do. There are so many of them, so the driver uses the least amount of memory needed, and you only include what is needed. Anyway, go ahead and try to build some of the demo samples and see if you can get them to work. Of course, they will not support the joysticks; we will have to add that later.

You will also notice a folder called **sample_images** this is part of the “bitmap” display support this driver has, which is pretty cool. The folder has a 128x64 sample .BMP file named **microbit_logo.bmp** along with an encoded raw byte version of it named the same without the .bmp extension. This is what the driver expects when you want to display a bitmap on the screen.

Additionally, there is a Python command-line tool named **bitmap_converter.py** in the folder to perform the conversions of bitmaps, along with an online tool you are supposed to use. This is all a bit much and too contrived for my taste, so I have written an online Javascript tool that does all the conversions and generates a single file for use by the driver in a single click. We will get to that later in this section, but for now, if you’re interested, please review the docs on GitHub, they cover the bitmap display support. That said, for now, let’s try using the driver to make a couple of simple demos and try some of the functions.

2.4.2 Fizban99 OLED Driver API Review

Let’s take a look at the most important functions and their usage. If you want to get any of the code snippets to work, make sure you include all the referenced API source modules in your project. This will make sure each program builds and you can upload it to the micro:bit.

```
initialize()
```

This is the first function you must call before using the driver library. It resets the display.

```
clear_oled()
```

This function clears the display to all black. It’s typically used after a call to **initialize()** or whenever you want to clear the display.

```
show_bitmap( filename )
```

The **show_bitmap()** function displays a pre-encoded raw data bitmap named **filename** on the OLED. The image has to be encoded as described previously as 1-bit 128x64 pixels (we will cover a full example of this process later in this section). Simply include the encoded bitmap in the project (it will get baked into the .HEX and loaded into the micro:bit). Here’s an example of displaying the sample image that comes with the driver in the **sample_images** folder.

Code Snippet 2.7 - Text Printing Example.

```
from ssd1306 import initialize, clear_oled
from ssd1306_bitmap import show_bitmap

initialize()
clear_oled()
show_bitmap("microbit_logo")
```

Make sure to include, along with the **main.py** code from the snippet in your project, the following files for this to build:

ssd1306.py	- The main driver file.
ssd1306_bitmap.py	- Bitmap display function.
microbit_logo	- Encoded bitmap data found in sample_images\ folder.

Figure 2.10 - The micro:bit logo displaying on the OLED display.



A copy of this demo is in the **Downloads** file tab named **Fizban99_MicrobitLogo_Display_Demo_01.hex** for your convenience. Next up the pixel plotting.

```
set_px(x, y, color, draw=1)
```

The **set_px()** function sets the pixel at position x, y (using the scaled 64 x 32 coordinate system for speed) with the corresponding color (0 black or 1 white). If the optional parameter **draw** is set to 0 the screen will not be refreshed until **draw_screen()** is called at a later time, since multiple screen refreshes can be time consuming. This allows setting different pixels in the frame buffer without refreshing the screen, and finally refreshing the display with the content of the buffer at the end.

Code Snippet 2.8 - Setting Pixels in the Frame Buffer with Draw_Screen() Buffer Write.

```
from ssd1306_px import set_px
from ssd1306 import draw_screen, initialize, clear_oled

initialize()
clear_oled()
set_px(10,10,1)
set_px(20,20,0,0)
draw_screen()
```

The snippet literally draws two pixels on the screen, so no screenshot, but the pre-built demo is named **Fizban99_Pixel_Demo_01.hex** If you want to see this code in action, The next function allows you to retrieve or read pixels from the frame buffer.

```
get_px(x, y)
```

The function **get_px()** returns the color of the given pixel (0 black, 1 white)

Code Snippet 2.9 - Reading a Pixel from the Frame Buffer.

```
from ssd1306 import initialize, clear_oled
from ssd1306_px import get_px

initialize()
clear_oled()

# read the pixel from position 10,10
color=get_px(10,10)
```

There is no demo for this snippet, so you will have to type it yourself if you want to try it. However, you might be asking, “Why do you need to read pixels that you wrote?” Well, in a game or simulation, you may want to read pixels for collision detection, or to see if there are any lit pixels around a point, for a nearest neighbor algorithm like **Conway’s “Life”** simulation or other cellular automata type processing. Next, let’s look at text printing support.

```
add_text(x, y, text, draw=1)
```

The function **add_text()** prints the string **text** at the row **x** and column **y**. The screen is divided into 12 columns and 5 rows, so it’s very small. If the optional **draw** parameter is set to 0 the screen will not be refreshed until **draw_screen()** is called. This allows writing different rows in the buffer without refreshing the screen, and finally refreshing the display with the content of the frame buffer. Below is a code snippet to print text.

Code Snippet 2.10 - Printing Text on the Screen with Fizban99 Driver.

```
from ssd1306 import initialize, clear_oled
from ssd1306_text import add_text

initialize()
clear_oled()
add_text(0, 2, "Hello, world")
```

You can find the code snippet in the **Downloads** master .ZIP in the location **\Demos\Fizbann99_Demos\Fizban99_Text_Demo_01.hex**. Figure 2.11 shows a screen shot of this demo in action.

Figure 2.11 - Printing Text with the Fizban99 Driver.



Next is the “stamp” bitmap support, which basically is like a sprite or small bitmap that can be drawn on the OLED.

```
create_stamp( img )
```

Creates a stamp from an **Image** object (built in micro:bit type used for LED display). You can find a list of these pre-defined bitmaps here:

<https://microbit-micropython.readthedocs.io/en/latest/tutorials/images.html>

For example, there is a Ghost and PacMan bitmap we can use for a little demo:

```
Image.PACMAN
Image.GHOST
```

A “stamp” is just a set of bytes that will be used to print the image on the OLED display. The function transforms any LED value different from 0 to 1. A stamp is defined with 5 columns of 8 pixels each, so a stamp occupies 5 bytes of memory and can also be defined as a byte array of 5 bytes. In other words, the stamp is 5 bytes long, but in column-major form: the first byte defines 8 vertical pixels of column 0, then the next byte 8 pixels of column 1, and so forth. If the stamp has been created from an **Image**, the stamp will be created by centering the image. This command is used in combination with `draw_stamp`

```
draw_stamp(x, y, stamp, color, draw=1)
```

Draws the stamp on the screen at the pixel position x, y. The stamp will be drawn using logical OR if color is 1 and logical AND NOT if color is 0, effectively removing the stamp when color=0. Below is a code snippet that draws a PacMan chasing a ghost on the OLED using the built-in **Image.PACMAN** and **Image.GHOST** objects.

Code Snippet 2.11 - PacMan Chasing a Ghost using “Stamps”.

```
from ssd1306 import initialize, clear_oled
from ssd1306_stamp import draw_stamp
from ssd1306_img import create_stamp
from microbit import Image, sleep

def move_stamp(x1, y1, x2, y2, stmp):
    draw_stamp(x1, y1, stmp, 0, 0)
    draw_stamp(x2, y2, stmp, 1, 1)

initialize()
clear_oled()
pacman = create_stamp(Image.PACMAN)
ghost = create_stamp(Image.GHOST)

pacmanX = 0
pacmanY = 16

ghostX = 16
ghostY = 16

while True:
    draw_stamp(pacmanX, pacmanY, pacman, 0, 0)
    draw_stamp(ghostX, ghostY, ghost, 0, 0)

    pacmanX = pacmanX + 1
    ghostX = ghostX + 1

    draw_stamp(pacmanX, pacmanY, pacman, 1, 1)
```

```
draw_stamp(ghostX, ghostY, ghost, 1, 1)
```

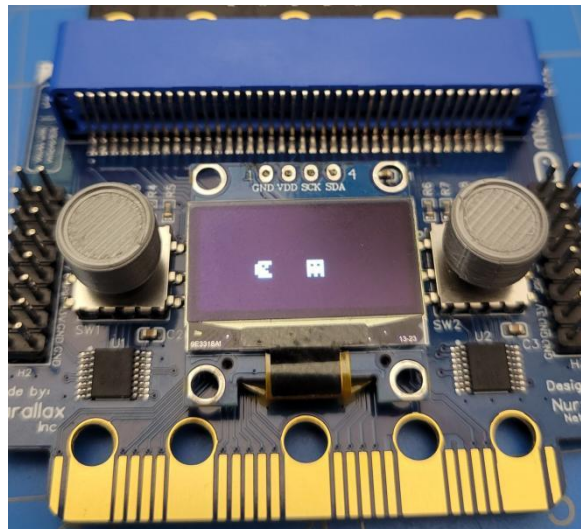
```
if (ghostX > 64 - 8):  
    clear_oled()  
    ghostX = 0
```

```
if (pacmanX > 64 - 8):  
    clear_oled()  
    pacmanX = 0
```

```
sleep(50)
```

You can find the code snippet in the **Downloads** master .ZIP located in **\Demos\Fizban99_Demos\Fizban99_Stamp_Demo_01.hex**. Figure 2.12 shows a screenshot of this demo in action.

Figure 2.12 - The Stamp Demo Drawing a PacMan Chasing a Ghost Forever.



Additionally, there is a little graphics “hack” when drawing a stamp in the demo above. When using the **draw_stamp()** function, it internally clears the contents of the screen just before the first column of the stamp and the contents of the screen just after the last column of the stamp is also redrawn. This is done to allow using a function like this below to perform a simple movement of a stamp without re-drawing the entire frame buffer:

```
def move_stamp(x1, y1, x2, y2, stmp):  
    draw_stamp(x1, y1, stmp, 0, 0)  
    draw_stamp(x2, y2, stmp, 1, 1)
```

The ***move_stamp()*** function above removes a stamp at position x1,y1 and redraws it at position x2, y2. Note that the first ***draw_stamp()*** does not refresh the screen. The screen is only refreshed once, with the second ***draw_stamp()***. If the stamp is 5x5 and it is centered within the 8x7 area, the stamp will be properly updated if the distance between the two coordinates is one pixel. In other words, you can only move the stamp 1-pixel at a time with this trick; otherwise, you will see a bitmap “trail”.

2.4.3 The Full Screen ***show_bitmap()*** Function Revisited

Previously, we saw the ***show_bitmap()*** function used to draw a pre-made bitmap of the micro:bit logo. However, we didn’t create the bitmap; we just used the source from the driver files found in Fizban99’s driver folder **sample_images** folder. But, if you reviewed the documentation on the GitHub and read my brief description of the format, you realize that you must use a bitmap that is black and white, 1-bit in other words, with dimensions 128x64, and this image must be converted by an online tool (referenced in the GitHub as well as then post converted by a Python tool provided by Fizban99 to the final raw, binary, byte stream expected by the ***show_bitmap()*** function. This is a lot of work, steps, and prone to errors, so I wrote a single online Javascript program that does it all for you! The program is hosted online here:

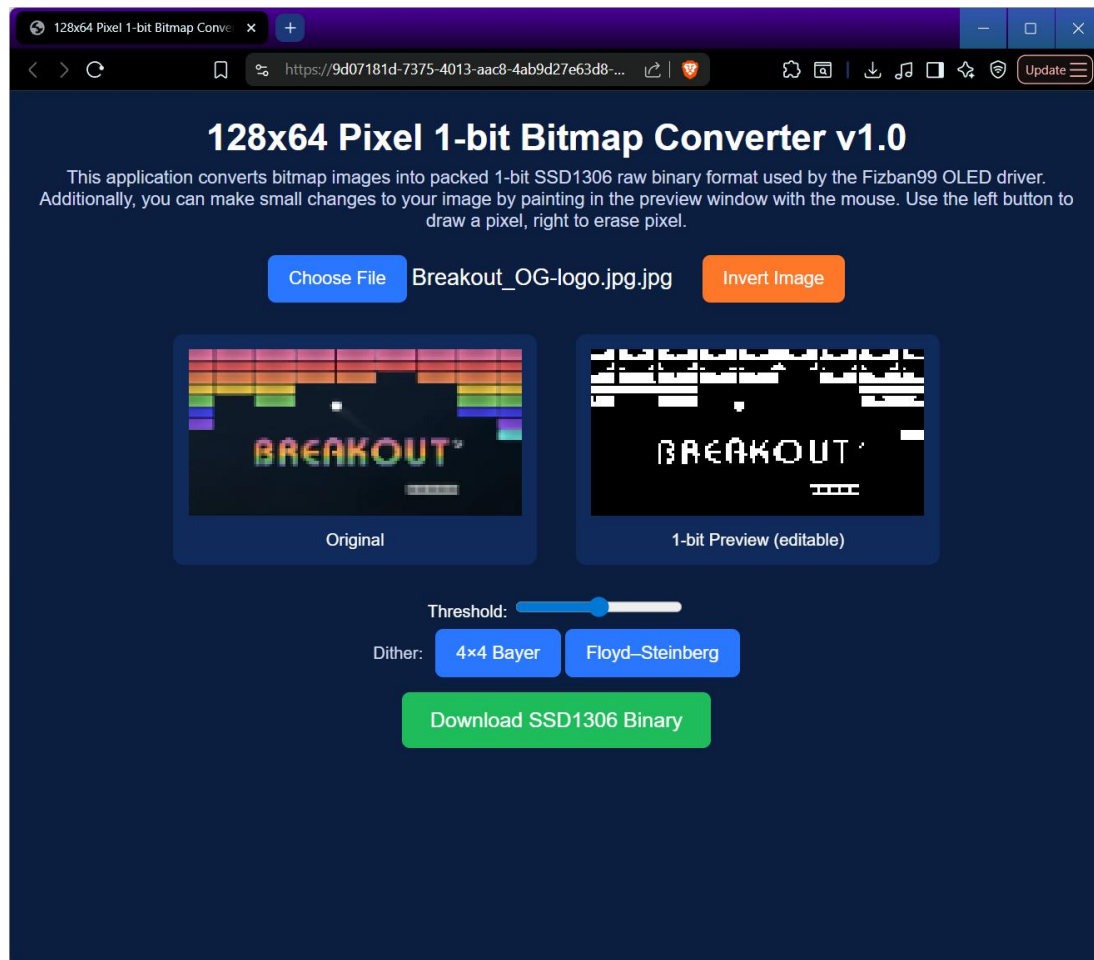
<https://bitmap-converter.replit.app/>

But, if the hosting fails, I have provided you with a single .HTML file in the main **Demos.zip** file in the GitHub repo at:

<https://github.com/NurveNetworksLLC/MicrobitOLEDBadge>

The files are located in the sub-folder **BitmapImageConverter**. Simple direct your browser to the **index.html** file within it, and you are in business. Either way, you should see something similar to Figure 2.13 below, which shows the online file converter in action.

Figure 2.13 - The Online File Format Converter to Generate Raw Binary Byte Data for Fizban99 `show_bitmap()` function.



The application is very simple to use; you provide it with a 128x64 pixel bitmap (the tool supports .PNG, .BMP, and .JPG), ideally in 1-bit monochrome. If not, the tool will autoscale the image, as well as convert it to 1-bit monochrome for you. Additionally, you can use the Threshold and Dithering options to “try” to make the image recognizable from the original. For example, you can see how I found a classic “Break Out” bitmap and then converted it. Also, you can actually draw in the preview window with the mouse to touch up the image. When you are happy with the final image, simply press the green **<Download SSD1306 Binary>** at the bottom of the page, and save the file. I suggest giving it an extension like .RAW or .BIN, so as to emphasize its encoded data.

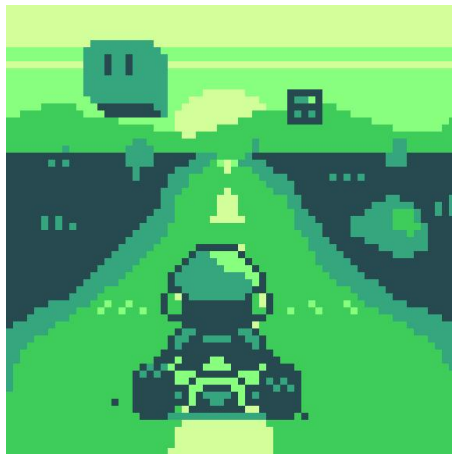
To use the generated bitmap file in your program, you simply include this file in your program with the other Python files. Then the micro:bit IDE will bake the data into the final .HEX firmware, and when you download the .HEX file, your program’s file system will be able to access this file as if it were on a disk. And the `show_bitmap()` function does just this.

TIP

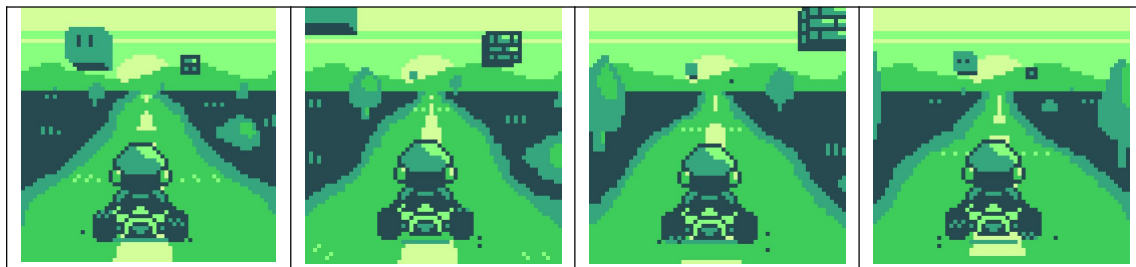
If you are looking for a good pixel art program to work with bitmaps, I suggest Krita, it's very powerful and has a relatively easy learning curve.

Now that you have the tools to convert any 128x64 pixel bitmap into the proper format for display, let's make a demo that shows what can be done with this show bitmap function. I found a few frames of Mario Kart as a .GIF on www.giphy.com from the artist "dan.bahia.dan", he has some other cool gifs as well:

https://giphy.com/dan_bahia_dan



Once the .GIF was downloaded, it was split into 4 single frames with the tool found here at <https://ezgif.com/> -- the results shown below:



Next, each frame was cropped down to 128x64 in Krita and then saved. The results are (4) clean 128x64 1-bit monochrome frames. Then, one by one, they were loaded into the Javascript conversion tool and output (4) frames named **frame_0.bin**, **frame_1.bin**, **frame_2.bin**, **frame_3.bin** respectively. You can find all these files in the **Downloads** folder in the following directory:

\Demos\AnimationDemo\Super_Mario_PixelArt_GIF_dan.bahia.dan

With the final (4) raw, binary byte files in hand, now we can make a little demo that flips through them frame by frame and creates a really cool 3D effect on the micro:bit badge. The source code is shown below in the snippet and can be found in the **Downloads** folder:

\Demos\AnimationDemo\fizban99_FullscreenAnimationDemo_01.hex

Code Snippet 2.12 - Full-Screen Mario Kart Animation Demo.

```
from ssd1306 import initialize, clear_oled
from random import randint
from ssd1306_bitmap import show_bitmap
from microbit import sleep

initialize()
while True:
    show_bitmap("frame_0.bin",0)
    sleep(100)
    show_bitmap("frame_1.bin",0)
    sleep(100)
    show_bitmap("frame_2.bin",0)
    sleep(100)
    show_bitmap("frame_3.bin",0)
    sleep(100)
```

As you can see, there is very little to the demo, just load the files and display them one after another. There is one thing I want to bring your attention to, though. The **show_bitmap()** function natively clears the OLED before displaying the image, and we don't want this because it's too slow and it disturbs the clean animation. So I modified the show_bitmap by adding an extra parameter that allows you to request a screen clear or not (0 - no clear, 1 - clear). Below is the hacked version of the function the demo uses:

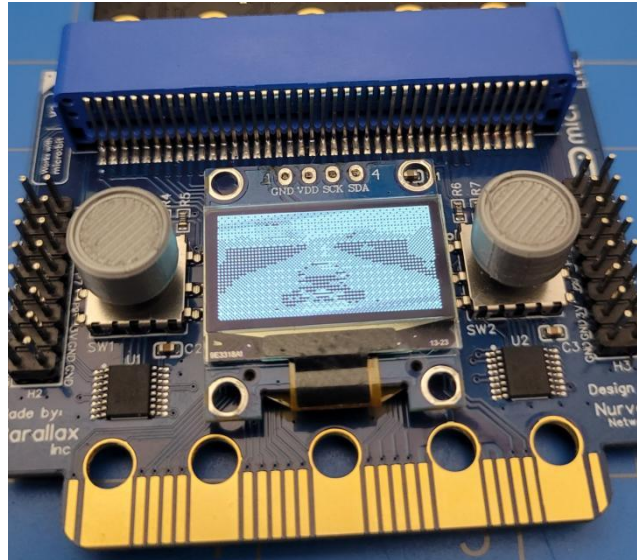
```
from microbit import i2c
from ssd1306 import command, set_zoom, set_pos, ADDR

# added last parameter to support clear/no clear
def show_bitmap(filename, clear=1):
    set_pos()
    if (clear):
        command([0xae])
    with open(filename, 'rb') as my_file:
        for i in range(0, 16):
            i2c.write(ADDR, b'\x40' + my_file.read(64))
    set_zoom(0)
    command([0xaf])
```

I made this modification only in the .HEX file for this demo, but you might want to use this in your demos if you want to have control over the clearing of the OLED by

default. With all that in mind, and assuming you have loaded the program into your micro:bit with the OLED badge plugged in, you should see something like that shown in Figure 2.14.

Figure 2.14 - Bitmap Animation Demo Running Full Screen.



The single image is hard to make out due to the dithering, but it looks much better when running and animating all the frames of animation. This wraps up our coverage.

3.0 - Reading the Joysticks

The joysticks are composed of two (2) 5-way momentary switches (Right, Left, Up, Down, and center for Fire) located on either side of the OLED. They are labeled **SW1** and **SW2**. As mentioned in the paragraphs above, the joystick switches are routed to a pair of 74LV165A parallel to serial shift registers, which are controlled via GPIO pins **P8**, **P9**, and **P16** with the following signals.

Table 3.1 - Joystick Signals and Pinouts.

GPIO Pin	Schematic Name	Function
P8	DPAD_LATCH	Tied to the 74LV165's shift/load (SH /LD) pins; selects whether clocking will latch the parallel inputs or shift the bits out.
P9	DPAD_CLK	Edge-triggered LOW to HIGH, clocks both shift registers during a latch or shift operation.
P16	DPAD_DOUT	Streams the 16 serial bits out representing the switch states for both joysticks. This signal is read into GPIO P16 bit by bit and assembled into a 16 bit word representing the joystick button states in the following order in hardware: (msb) Right Joystick Left Joystick (lsb) [1,1,1, R,L,F,D,U 1,1,1, R,L,F,D,U] In software, the driver can re-arrange the raw bits in any order it wishes, return them as an array/list, etc.

We have developed demos and sample drivers for you to read the joysticks in **both micro:bit Makecode Python** (which can be reloaded as block code using the editor) as well as more formal **micro:bit MicroPython** using the **python.microbit.org** editor. Later in the document, you will see various demos that use both the OLED display and the joystick, but right now, let's review the set of functions that we have provided to read the joysticks. You can use these to write your own, optimize them, or learn how low-level GPIO programming is done, even if you just want to use them.

In the following section, we will review the code snippets from the micro:bit MicroPython version. The Makecode version is nearly identical except for the declaration of GPIO pins, and the write and read digital pin method syntax is a little different.

The first step in communicating with the joysticks (or writing any GPIO level code) is importing the GPIO pins, as well as any other modules needed.

Code Snippet 3.1 - Joystick GPIO Pin Initialization

```
# imports needed
from microbit import pin8, pin9, pin16, sleep, display

# --- Pin mapping (MakeCode / PXT) ---
PIN_LATCH = pin8    # /PL (active low)
PIN_CLK   = pin9    # CP (rising edge shifts)
PIN_DATA  = pin16   # /Q7 (inverted: pressed -> 1)

# Idle states
PIN_LATCH.write_digital(1) # shift mode idle
PIN_LATCH.write_digital(0) # clock idle low
```

Referring to code snippet 3.1, some imports are made to access the GPIO pins, as well as the sleep functions and 5x5 LED display (for status outputs). Next, we create the pin variables for the latch, clock, and data pins connected to the 74LV165 shift registers. Finally, we set the latch and clock pins to a known state; this is very important and a good habit when controlling external hardware and ICs.

Next, let's take a look at some of the helper functions that clock in the serial data, rearrange it, display it, and so forth. The next function shown in snippet 3.2 is a latch macro function, which is called to latch the joystick switch states into the shift registers.

Code Snippet 3.2 - Shift Register Latch Macro Function.

```
# This function simply pulses the latch pin
def pulse_latch():
    # Latch current button states into the 165s
    PIN_LATCH.write_digital(0)
    sleep(1)
    PIN_LATCH.write_digital(1)
```

Now, we have everything needed to read the joystick switch states and stream them in serially. The next code snippet 3.3 lists ***read_two_bytes()*** which performs this task.

Code Snippet 3.3 - Joystick Byte Read Function.

```
def read_two_bytes():
    """
    Read 16 bits from the daisy-chained 74LV165s into two bytes:
    First 8 clock cycles -> RIGHT byte (D7..D0)
    Next 8 clock cycles -> LEFT byte (D7..D0)
    We sample DATA before the rising edge; shift on rising edge.
    """
    pulse_latch()

    right_byte = 0
    left_byte = 0

    # Read RIGHT (first 8 bits, MSB-first -> D7..D0)
    for i in range(8):
        bit = PIN_DATA.read_digital() & 1 # 1 = pressed (via /Q7)
        right_byte = (right_byte << 1) | bit
        PIN_CLK.write_digital(1)
        PIN_CLK.write_digital(0)

    # Read LEFT (next 8 bits)
    for i in range(8):
        bit = PIN_DATA.read_digital() & 1
        left_byte = (left_byte << 1) | bit
        PIN_CLK.write_digital(1)
        PIN_CLK.write_digital(0)

    # After these loops:
    # right_byte bits 7..0 == D7..D0 for RIGHT chip
    # left_byte bits 7..0 == D7..D0 for LEFT chip
    return left_byte, right_byte
```

The function starts by latching the two joystick switch states into the 74LV165 shift registers. Then once the data is latched, it is shifted out bit by bit into the DPAD_DOUT GPIO at pin 16. This is done in two 8-bit chunks: one loop for the left joystick and the other for the right. And as this process is performed, each bit is shifted using bit operations to the left of the words **left_byte** and **right_byte** respectively. When the function is complete, it returns the two values in a single return.

With these functions, you can simply make the call below to read the joysticks:

```
# read joysticks into two variables
left, right = read_two_bytes()
```

However, the problem is that the button states are bit encoded, so you have to do bitwise tests to check each button on the left and right variables. Nothing wrong with this, but we have created a couple more helper functions that might be easier to use. The first is a bit test function that returns if a bit is set in a word, it's shown below in snippet 3.4.

Code Snippet 3.4 - Bit Test Helper Function.

```
def btest(byte, bit_index):
    # Helper function, returns 1 if bit_index is set in byte, else 0.
    if (byte >> bit_index) & 1:
        return 1
    else:
        return 0
```

The helper function ***btest(byte, bit_index)*** simply takes the byte or word you want tested, and the bit index (0 - lsb) you want tested, then returns 1 if the bit is set, 0 otherwise. This function allows us to quickly interrogate the left and right bytes to see if a bit is set, and hence a button (directional pressed). For example, if you wanted to read the joysticks and test if either “fire” button was pressed, you could write the following code shown in snippet 3.5.

Code Snippet 3.5 - Reading the Joysticks and Testing for Fire Buttons.

```
# read both joysticks in
left, right = read_two_bytes()

# next check if the fire buttons are both down?
if (btest(left, 2) and btest(right, 2)):
    # do this...
```

And that’s it - you’ve initialized and read the joysticks! Of course, we can always add more helper functions, and you might not like bit testing, so snippet 3.6 lists a final helper function ***decode_udflr_from_byte(byte_val)*** that unpacks/splits the bits into an array/list and returns it.

Code Snippet 3.6 - Joystick Raw Bit Data Decoder Function.

```
# Decode helpers: return 0/1 for U,D,F,L,R in fixed order
def decode_udflr_from_byte(byte_val):
    # byte_val bit mapping: bit0 = D0, bit1 = D1, ..., bit4 = D4
    up = btest(byte_val, 0) # D0
    down = btest(byte_val, 1) # D1
    fire = btest(byte_val, 2) # D2
    left = btest(byte_val, 3) # D3
    right = btest(byte_val, 4) # D4
    # return list [U, D, F, L, R]
    return [up, down, fire, left, right]
```

The code in snippet 3.6 doesn’t really do much other than split the bits out into a list, and changes their order a bit to make things easier to deal with as arrays with integer indices. For example, if we wanted to read the joysticks and then increase a volume variable (or servo position? etc.) based on the left joystick’s right and left motion, you could write the following code.

Code Snippet 3.7 - Example of Reading Joystick to Set a Volume Variable.

```
# holds the value of our volume variable
volume = 0

# read both joysticks in
left, right = read_two_bytes()

# decode the left bit packed data
left_list = decode_udflr_from_byte( left )

# is the user pressing the left joystick left?
if (left_list[ 3 ]):
    volume = volume - 1 # decrease volume
# is the user pressing the left joystick left?
elif (left_list[ 4 ]):
    volume = volume + 1 # increase volume
```

That wraps it up for reading the joysticks, hopefully, you have a good understanding of exactly how the hardware works, and how bit operations and GPIO operations are used to read the data. There are full demos in section **4.0 - Demos and Examples**, in both Makecode Python and micro:bit MicroPython formats to help you get started.

3.1 Joystick Driver Library

We have taken the GPIO setup, and the main joystick reading and decoding functions, and placed them in a library for your convenience. There are two versions of the library: one for the Makecode version of Python and one for the “real” MicroPython version. The libraries can be found in the Downloads tab with the names:

Makecode “Python” Version: microbit_Makecode_Python_Joystick_Driver_01.py

MicroPython Version: microbit_MicroPython_Joystick_Driver_01.py

For example, if you are using MicroPython with the site

<https://python.microbit.org/> or Thonny, simply include either the driver library in your project then import it like this at the top of you main Python program:

```
from microbit_MicroPython_Joystick_Driver_01 import *
```

Then the two functions we wrote in the joystick coverage above are available and used as follows.

To read the joysticks at the same time:

```
left, right = read_two_bytes()
```

Then to decode each joystick byte into a list of booleans [up, down, fire, left, right]:

```
left_list = decode_udflr_from_byte( left )
```

```
left_list = decode_udflr_from_byte( right )
```

And that's it! Of course, you can directly use the byte-encoded **left** and **right** packed bytes as well if you want to deal with bit manipulations.

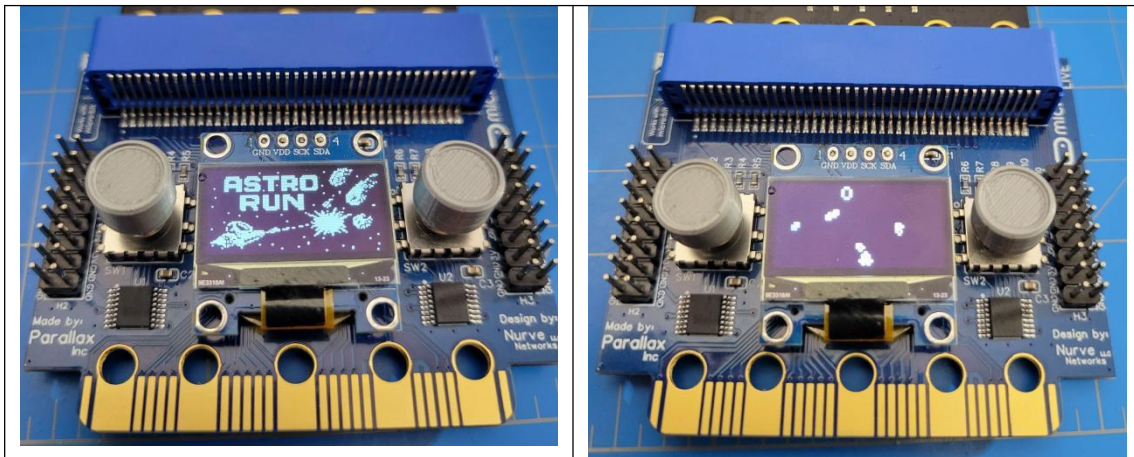
4.0 - Demos and Examples

In this section, each demo and example is listed out with the filename and location in the master web **Downloads** link files as **Demos.zip**. First, let's take a look at some of the games based on the Fizban99 driver.

4.1 Astro Run

The first game we are going to take a look at is called **"Astro Run"**. This game is shown in Figure 4.1 below. The game is based on the "Asteroids" demo found in the Fizban99 driver folder `\samples\asteroids\` which itself is based on another game called **"BitFlyer"**.

Figure 4.1 - micro:bit Astro Run Screen Shots.



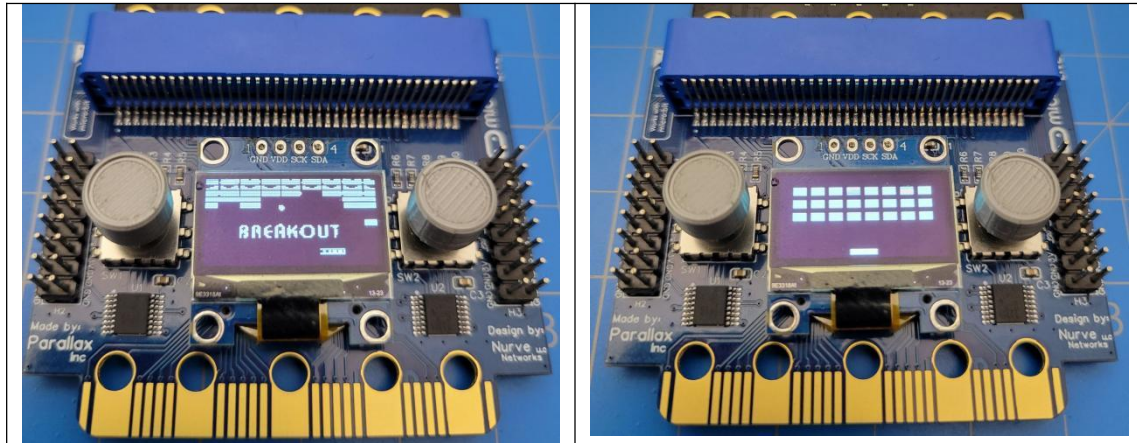
I added joystick support to **"Astro Run"**, along with actual gameplay. Start the game by pressing down the joystick center "fire" button. You control your ship with the left joystick (left and right), and fire by simply pressing the right joystick up, and even rapid fire. As you shoot the asteroids, you get points. You die, the game ends, press the fire buttons again to re-start. The micro:bit MicroPython .hex source file is located here:

`\Demos\AstroRun\fizban99_AstroRun_JoystickSupport_04.hex`

4.2 BreakOut

The next game is an attempt at Atari “**BreakOut**”. I ran into memory limitations pretty quickly, so the game is not complete, but it’s a good start. Nevertheless, another good example of what you can do with the Fizban99 driver. Figure 4.2 below shows the game in action.

Figure 4.2 - micro:bit “BreakOut” Clone Screen Shots.



The game is located here:

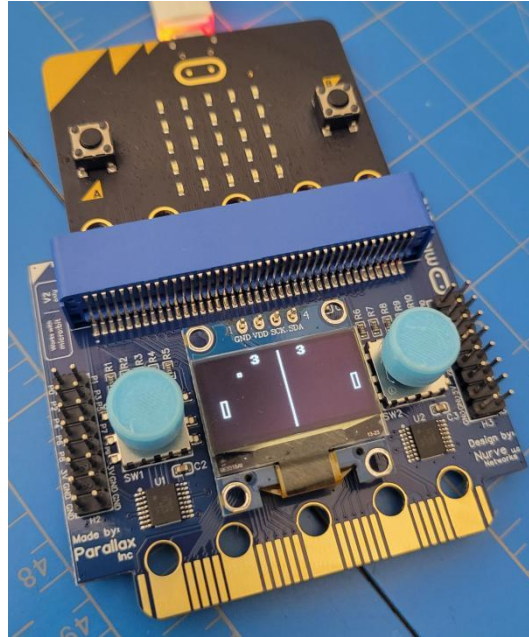
`\Demos\BreakOut\fizban99_Breakout_JoystickSupport_02`

Again, this game is designed for the micro:bit MicroPython IDE, not Makecode block editor.

4.3 Pong Demo

This demo is a little clone of the classic Atari Pong!. It supports two players, and uses the PiicoDev OLED driver; it's VERY slow! I suggest trying to rewrite it with the Fizban99 driver, which is much faster. Figure 4.3 below shows the game in action.

Figure 4.3 - Pong! Demo in Action.



The game is located here:

`\Demos\Pong`

4.4 General Joystick Demos

Below is a list of extra joystick demos for both Makecode Python and MicroPython.

This demo uses the PiicoDev OLED driver. The demo uses the left and right joystick to move a rectangle around on the screen. The left joystick to move, right joystick to resize rectangle. This demo runs via the micro:bit MicroPython IDE.

`\Demos\JoystickDemos\Microbit_MicroPython_Badge_JoystickBoxTest_01.hex`

Next, there is a Makecode version of the same demo that runs in the Makecode IDE. This time, it uses the PXT OLED driver, but the functionality and controls are the same.

`\Demos\JoystickDemos\microbit-MAKECODE_Python_Badge_JoystickBoxTest_01.hex`

5.0 - External Resources

In this section you will find a number of useful websites, links, and other resources that will help you program and take advantage of the micro:bit OLED Badge.

https://soldered.com/productdata/2022/03/Soldered_SSD1306_datasheet.pdf
https://soldered.com/productdata/2022/03/Soldered_SSD1315_datasheet.pdf
<https://www.ti.com/lit/ds/symlink/sn74lv165a.pdf?ts=1763098436746>

OLED driver links, GitHub, etc.

<https://www.youtube.com/watch?v=xEcLUxhMMDY>

<https://tech.microbit.org/hardware/edgeconnector/>
<https://tech.microbit.org/hardware/schematic/>

6.0 - Pin Definitions and Ratings

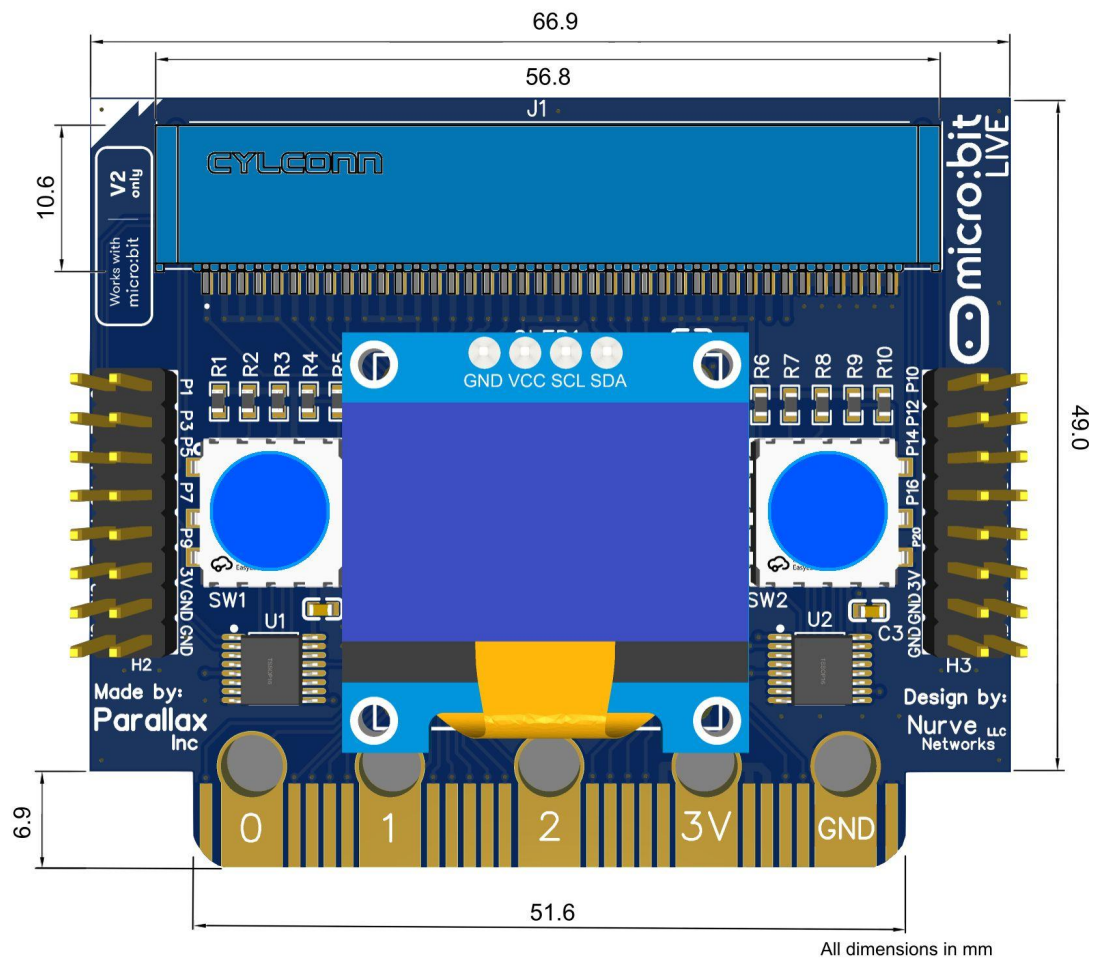
(Edge Interface and H2/H3 2x8 0.1" Breakout Headers)

Table 6.1 - Pin Definitions and Ratings.

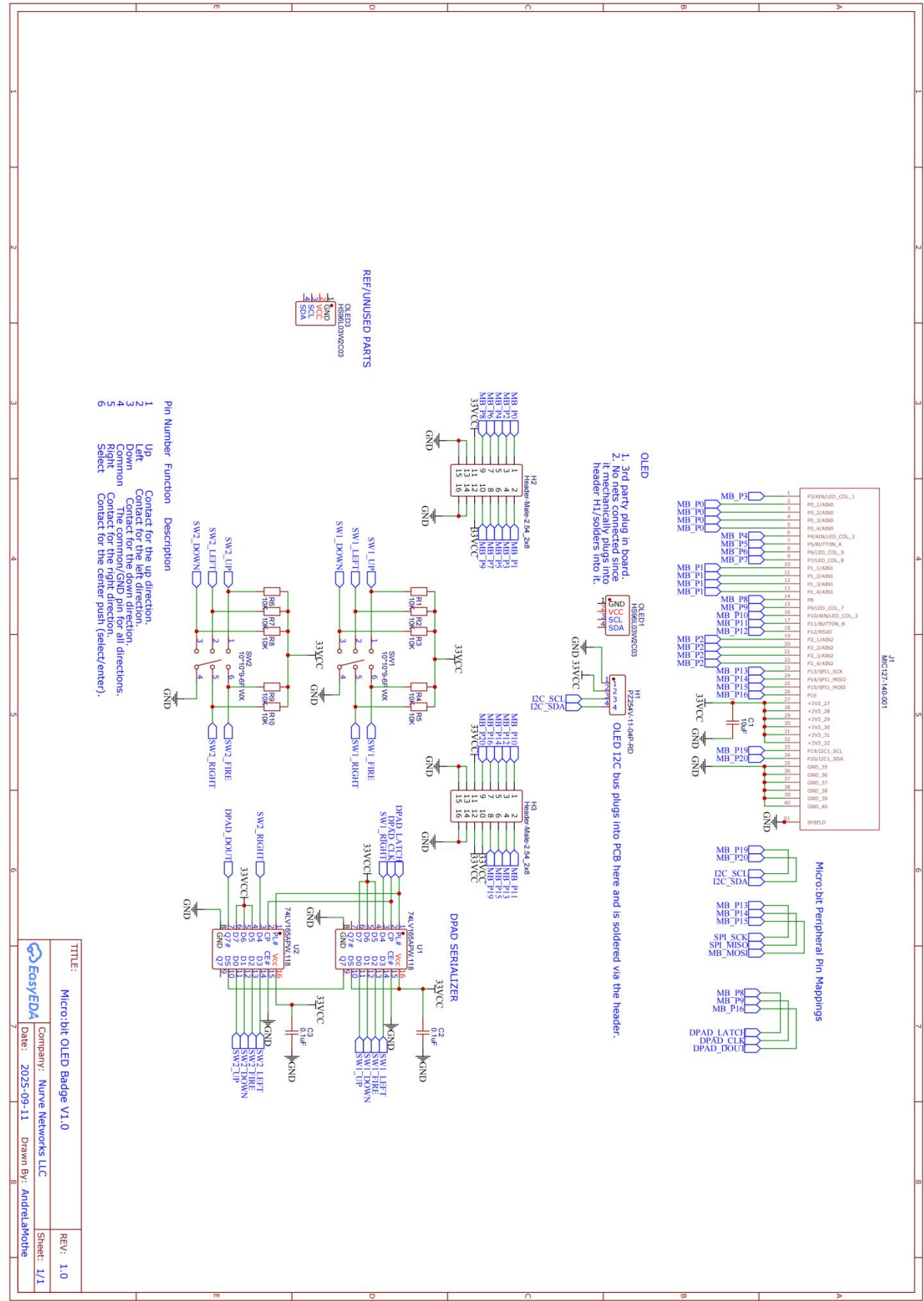
Label	Type	Function
P0	IO	GPIO (general purpose digital input/output) with ADC (analog to digital converter).
P1	IO	GPIO with ADC.
P2	IO	GPIO with ADC.
P3	IO	GPIO shared with LED Col 3 (v2) of the LED screen; can be used for ADC and digital I/O when the LED screen is turned off.
P4	IO	GPIO shared with LED Col 1 (v2) of the LED screen; can be used for ADC and digital I/O when the LED screen is turned off.
P5	IO	GPIO shared with Button A, to trigger or detect a button "A" click externally. This pin has a pull-up resistor, which means that by default it is at voltage of 3V. To replace button A on the micro:bit with an external button, connect one end of the external button to pin 5 and the other end to GND. When the button is pressed, the voltage on pin 5 is pulled down to 0, which generates a button click event.
P6	IO	GPIO shared with LED Col 4 (v2) of the LED screen; can be used for digital I/O when the LED screen is turned off.
P7	IO	GPIO shared with LED Col 2 (v2) of the LED screen; can be used for digital I/O when the LED screen is turned off.
P8	IO	GPIO shared by the OLED adapter as output DPAD_LATCH to latch the state of joystick buttons during serialization of joysticks. It can be used as input or output while not reading the joysticks.
P9	IO	GPIO shared by the OLED adapter as output DPAD_CLK to clock the bits out of the joystick shift registers into the micro:bit GPIO P16 which then is read by software. It can be used as input or output while not reading the joysticks.
P10	IO	GPIO shared with LED Col 5 (v2) of the LED screen; can be used for ADC and digital I/O when the LED screen is turned off.
P11	IO	GPIO shared with Button B, to trigger or detect a click externally.
P12	IO	This GPIO pin has been reserved to provide support for accessibility.
P13	IO	GPIO that is conventionally used for the serial clock (SCK) signal of the 3-wire Serial Peripheral Interface (SPI) bus.
P14	IO	GPIO that is conventionally used for the Master In Slave Out (MISO) signal of the SPI bus.
P15	IO	GPIO that is conventionally used for the Master Out Slave In (MOSI) signal of the SPI bus.
P16	IO	GPIO shared by the OLED adapter to stream in data from the shift registers DPAD_DOUT signal. This GPIO must not be driven from the external interfaces, and must be in INPUT mode when the OLED adapter is plugged in. This pin is sometimes used as a chip select for SPI devices, so another GPIO must be used as chip select.
P19	IO	Clock signal (SCL) of the I2C bus communication protocol. Connected to OLED I2C bus at 0x3C (or 0x3D on some OLEDs).
P20	IO	Data line (SDA) of the I2C bus communication protocol. Connected to OLED I2C bus at 0x3C (or 0x3D on some OLEDs).
GND	Power	System ground from micro:bit.
3V	Power	3.3V power output directly from micro:bit. Maximum 90mA with micro:bit v1, Maximum 300mA with micro:bit v2. Do not use to power servo motors or high current accessories! NO reverse current or overload protection!

Mechanical Drawings and Dimensions - Appendix A

PCB Overall Size: 66.9 x 58.9 mm (2.64 x 2.31 in)



Electrical Schematic - Appendix B



Revision History

Version 1.1 - Original Release (7/7/26) - Written by Andre' LaMothe
Editing and general clean up of text.

Version 1.0 - Original Release (12/11/25) - Written by Andre' LaMothe